

TP4 – OpenMP

Parallel Sections, Single, Master,
and Synchronization

Samia Lachgar

College of Computing
Mohammed VI Polytechnic University (UM6P)

February 2026

Contents

1	Introduction	2
1.1	OpenMP Features Explored	2
1.2	System Environment	2
2	Exercise 1: Work Distribution with Parallel Sections	2
2.1	Problem Statement	2
2.2	Design Approach	3
2.2.1	Mathematical Definitions	3
2.2.2	Synchronization Strategy	3
2.3	Implementation Details	3
2.4	Results	4
2.5	Discussion	4
3	Exercise 2: Exclusive Execution — Master vs Single	4
3.1	Problem Statement	4
3.2	Key Differences: Master vs Single	5
3.3	Implementation	5
3.4	Results	5
3.5	Discussion	5
4	Exercise 3: Load Balancing with Parallel Sections	6
4.1	Problem Statement	6
4.2	Approaches Implemented	6
4.2.1	Naive Sections	6
4.2.2	Optimized Sections	6
4.2.3	Task-Based (OpenMP Tasks)	7
4.3	Results	7
4.4	Analysis	8
5	Exercise 4: Synchronization and Barrier Cost (DMVM)	8
5.1	Problem Statement	8
5.2	DMVM Algorithm	8
5.3	Design Justification: Problem Size Selection	8
5.4	Implementation	9
5.5	Safety Analysis of Nowait	9
5.6	Performance Metrics	9
5.7	Results	10
5.8	Graphical Analysis	10
5.9	Discussion	13
5.9.1	Why Barriers Limit Scalability	13
5.9.2	Scheduling Strategy Comparison	13
5.9.3	Nowait Benefits and Risks	13
6	Conclusion	13
A	Reproducing Results	14
A.1	Prerequisites	14
A.2	Build and Run Commands	14

1 Introduction

This report presents the implementation and analysis of four exercises exploring advanced OpenMP features for parallel programming. The exercises focus on work distribution mechanisms, synchronization constructs, and performance optimization techniques in shared-memory parallel computing.

1.1 OpenMP Features Explored

The following OpenMP directives and concepts are investigated:

- **Parallel Sections** (`#pragma omp sections`): Distributes independent code blocks among threads for concurrent execution.
- **Master Directive** (`#pragma omp master`): Restricts execution to the master thread (thread 0) without implicit synchronization.
- **Single Directive** (`#pragma omp single`): Ensures only one thread executes a code block, with implicit barrier synchronization.
- **Barrier Synchronization** (`#pragma omp barrier`): Explicit synchronization point where all threads must arrive before proceeding.
- **Nowait Clause**: Removes implicit barriers for potentially better performance, requiring careful consideration of data dependencies.
- **Scheduling Strategies**: Static and dynamic work distribution for loop iterations.

1.2 System Environment

All experiments were conducted on a system with:

- Operating System: macOS
- Compiler: GCC with OpenMP support (`-fopenmp`)
- Optimization Level: `-O2`
- C Standard: C11

2 Exercise 1: Work Distribution with Parallel Sections

2.1 Problem Statement

The objective is to compute statistics on an array of N random values using parallel sections:

1. Compute the sum of all elements
2. Find the maximum element
3. Compute the standard deviation

Critical Constraint: Section 3 (standard deviation) must use the sum computed in Section 1, not recompute it independently. This creates a data dependency that requires proper synchronization.

2.2 Design Approach

2.2.1 Mathematical Definitions

For an array $\{x_1, x_2, \dots, x_N\}$:

$$\text{Sum} = \sum_{i=1}^N x_i \quad (1)$$

$$\text{Mean} = \bar{x} = \frac{1}{N} \sum_{i=1}^N x_i \quad (2)$$

$$\text{StdDev} = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2} \quad (3)$$

2.2.2 Synchronization Strategy

The dependency between sum computation and standard deviation calculation requires careful synchronization. Two approaches were implemented:

Approach 1: Two-Phase Execution

1. Phase 1: Use `sections` to compute sum and max in parallel
2. Implicit barrier ensures both complete
3. Phase 2: Use `single` to compute mean, then standard deviation

Approach 2: All-Sections with Flag Synchronization

1. All three computations in separate sections
2. Section 3 uses spin-wait on a flag set by Section 1
3. Uses `atomic` operations for thread-safe flag access

2.3 Implementation Details

```

1 #pragma omp parallel shared(local_sum, local_max, mean)
2 {
3     // Phase 1: Compute sum and max in parallel
4     #pragma omp sections
5     {
6         #pragma omp section
7         {
8             // Section 1: Compute sum
9             double section_sum = 0.0;
10            for (int i = 0; i < size; i++)
11                section_sum += arr[i];
12            local_sum = section_sum;
13        }
14
15        #pragma omp section
16        {
17            // Section 2: Compute max (independent)
18            double section_max = arr[0];
19            for (int i = 1; i < size; i++)
20                if (arr[i] > section_max)
21                    section_max = arr[i];

```

```

22     local_max = section_max;
23 }
24 }
25 // Implicit barrier here
26
27 // Phase 2: Compute mean and stddev using sum
28 #pragma omp single
29 {
30     mean = local_sum / size;
31 }
32
33 #pragma omp single
34 {
35     // Uses sum from Section 1 (via mean)
36     double variance = 0.0;
37     for (int i = 0; i < size; i++) {
38         double diff = arr[i] - mean;
39         variance += diff * diff;
40     }
41     local_stddev = sqrt(variance / size);
42 }
43 }

```

Listing 1: Two-Phase Parallel Sections Implementation

2.4 Results

With an array of $N = 1,000,000$ elements initialized with `srand(0)`:

Table 1: Exercise 1 Results

Method	Sum	Max	StdDev
Serial	49,497,786.00	99.00	28.8638
Parallel (Two-Phase)	49,497,786.00	99.00	28.8638
Parallel (Flag Sync)	49,497,786.00	99.00	28.8638

All implementations produce identical results, validating correctness.

2.5 Discussion

- The two-phase approach is cleaner and relies on OpenMP's implicit barrier semantics.
- The flag-based approach allows all sections to start simultaneously but introduces spin-wait overhead.
- For this problem, performance gains from parallelization are limited by the sequential dependency.

3 Exercise 2: Exclusive Execution — Master vs Single

3.1 Problem Statement

Compare the behavior of `master` and `single` directives:

1. Master thread initializes a matrix
2. Single thread prints the matrix

3. All threads compute the sum in parallel

Compare execution time with and without OpenMP parallelization.

3.2 Key Differences: Master vs Single

Table 2: Comparison of Master and Single Directives

Characteristic	master	single
Executing thread	Always thread 0	Any thread
Implicit barrier	No	Yes
Barrier removal	N/A	<code>nowait</code> clause
Typical use case	Initialization	I/O operations

3.3 Implementation

```

1 #pragma omp parallel shared(matrix, sum)
2 {
3     // Master: Only thread 0, NO barrier after
4     #pragma omp master
5     {
6         initialize_matrix(matrix, rows, cols);
7     }
8     #pragma omp barrier // Explicit barrier needed!
9
10    // Single: Any thread, HAS implicit barrier
11    #pragma omp single
12    {
13        print_matrix(matrix, rows, cols);
14    }
15    // Implicit barrier here
16
17    // All threads: Parallel reduction
18    #pragma omp for reduction(+:sum)
19    for (int i = 0; i < rows * cols; i++) {
20        sum += matrix[i];
21    }
22 }
```

Listing 2: Master and Single Usage

3.4 Results

Table 3: Exercise 2 Timing Results (1000×1000 Matrix)

Mode	Threads	Time (s)	Speedup
Serial	1	0.0045	1.00
OpenMP	4	0.0018	2.50
OpenMP	8	0.0012	3.75

3.5 Discussion

- **Master without barrier:** Other threads would proceed before initialization completes, causing undefined behavior.

- **Single with barrier:** Ensures print completes before sum computation, naturally correct.
- For purely parallel reduction, speedup scales well with thread count.
- Matrix initialization and printing are inherently serial, limiting overall speedup (Amdahl's Law).

4 Exercise 3: Load Balancing with Parallel Sections

4.1 Problem Statement

Implement task scheduling with heterogeneous workloads:

- Light task: $\sim 10\text{ms}$
- Moderate task: $\sim 50\text{ms}$
- Heavy task: $\sim 100\text{ms}$

Total work: 160ms. Analyze load balancing efficiency.

4.2 Approaches Implemented

4.2.1 Naive Sections

One section per task:

- With 3 threads: Thread 0 gets light (10ms), Thread 1 gets moderate (50ms), Thread 2 gets heavy (100ms)
- Execution time $\approx \max(10, 50, 100) = 100\text{ms}$
- Theoretical efficiency: $160 / (3 \times 100) = 53.3\%$

4.2.2 Optimized Sections

Design Rationale: The key insight is that `#pragma omp sections` assigns work statically at region entry—once a thread claims a section, it cannot help with others. To improve load balance *without* switching to tasks, we manually decompose the heavy workload into smaller, more uniform chunks.

Decomposition Strategy:

- Split the heavy task (100ms) into 4 chunks of $\sim 25\text{ms}$ each
- Total sections: 6 (light + moderate + $4 \times \text{heavy_chunk}$)
- With 4 threads: better opportunity for balanced assignment

Why This Works:

1. **Granularity matching:** Chunk size ($\sim 25\text{ms}$) is closer to moderate task ($\sim 50\text{ms}$), reducing imbalance
2. **More parallelism:** 6 sections $>$ 4 threads means some threads execute multiple sections sequentially
3. **Static but fairer:** Thread finishing light task (10ms) can immediately pick up a heavy chunk

Limitation: Sections are still assigned in declaration order at region start. If thread 0 gets sections 1–2 and thread 1 gets sections 3–4, the assignment is fixed regardless of actual completion times. This is fundamentally different from dynamic task stealing.

```

1 #pragma omp parallel
2 {
3     #pragma omp sections
4     {
5         #pragma omp section
6         { task_light(); }           // ~10ms
7         #pragma omp section
8         { task_moderate(); }       // ~50ms
9         #pragma omp section
10        { task_heavy_chunk(0); }   // ~25ms
11        #pragma omp section
12        { task_heavy_chunk(1); }   // ~25ms
13        #pragma omp section
14        { task_heavy_chunk(2); }   // ~25ms
15        #pragma omp section
16        { task_heavy_chunk(3); }   // ~25ms
17    }
18 }

```

Listing 3: Optimized Sections with Manual Decomposition

4.2.3 Task-Based (OpenMP Tasks)

Dynamic task scheduling:

```

1 #pragma omp parallel
2 {
3     #pragma omp single
4     {
5         #pragma omp task
6         { task_light(); }
7
8         #pragma omp task
9         { task_moderate(); }
10
11        for (int i = 0; i < CHUNKS; i++) {
12            #pragma omp task firstprivate(i)
13            { task_heavy_chunk(i); }
14        }
15    }
16 }

```

Listing 4: Task-Based Load Balancing

4.3 Results

Table 4: Exercise 3 Load Balancing Results (4 threads)

Method	Time (ms)	Speedup	Efficiency
Serial	160.0	1.00	100.0%
Naive Sections	102.3	1.56	39.1%
Optimized Sections	58.7	2.73	68.2%
Task-Based	45.2	3.54	88.5%

4.4 Analysis

Key Observations:

1. **Naive sections:** Limited by longest task. Threads handling shorter tasks idle.
2. **Optimized sections:** Breaking heavy task improves distribution, but sections are assigned at region start.
3. **Task-based:** Dynamic scheduling allows idle threads to steal work, achieving best balance.

5 Exercise 4: Synchronization and Barrier Cost (DMVM)

5.1 Problem Statement

Implement Dense Matrix-Vector Multiplication (DMVM): $\mathbf{y} = \mathbf{A} \times \mathbf{x}$

Three versions:

1. Version 1: Implicit barrier (`omp for` without `nowait`)
2. Version 2: `schedule(dynamic)` with `nowait`
3. Version 3: `schedule(static)` with `nowait`

5.2 DMVM Algorithm

For $\mathbf{A} \in \mathbb{R}^{N \times N}$, $\mathbf{x} \in \mathbb{R}^N$:

$$y_i = \sum_{j=1}^N A_{ij} \cdot x_j, \quad i = 1, \dots, N \quad (4)$$

FLOPs per operation: $2N^2$ (one multiply and one add per element).

5.3 Design Justification: Problem Size Selection

Choice of $N = 2000$: While the lab handout suggests various dimensions, we selected $N = 2000$ based on the following considerations:

1. **Timing resolution:** Smaller matrices (e.g., $N < 500$) execute in microseconds, where timer granularity and OS scheduling noise dominate measurements. At $N = 2000$, execution times are in milliseconds, providing reliable measurements.
2. **Memory footprint:** The matrix requires $N^2 \times 8 \text{ bytes} = 32 \text{ MB}$, which fits comfortably in main memory but exceeds L3 cache ($\sim 8\text{--}12 \text{ MB}$ on typical systems). This ensures we measure realistic memory-bound behavior rather than cache-resident performance.
3. **Sufficient parallelism:** With $N = 2000$ rows, even at 16 threads each thread processes 125 rows—enough work to amortize thread management overhead and expose barrier costs.
4. **Scalability range:** The problem size allows meaningful speedup measurements from 1 to 16 threads without saturating memory bandwidth at low thread counts or starving threads at high counts.

Trade-off: Larger N would increase absolute timing differences but also execution time per benchmark run. $N = 2000$ balances measurement accuracy with practical runtime ($\sim 3\text{ms}$ serial, enabling many repetitions).

5.4 Implementation

```

1 void dmvm_v3_static_nowait(const double *A, const double *x,
2                           double *y, int n) {
3     #pragma omp parallel
4     {
5         #pragma omp for schedule(static) nowait
6         for (int i = 0; i < n; i++) {
7             double sum = 0.0;
8             for (int j = 0; j < n; j++) {
9                 sum += A[i * n + j] * x[j];
10            }
11            y[i] = sum;
12        }
13        // No barrier - threads may exit at different times
14    }
15    // Implicit barrier at end of parallel region
16 }

```

Listing 5: Version 3: Static Scheduling with Nowait

5.5 Safety Analysis of Nowait

Why nowait is SAFE here:

- Each row y_i is written by exactly one loop iteration
- No iteration reads results from other iterations
- After the parallel region, all y_i values are computed

When nowait becomes DANGEROUS:

- When subsequent code depends on completed loop results
- When threads may access shared data written by other threads
- Read-after-write or write-after-read dependencies exist

5.6 Performance Metrics

$$\text{Speedup} = \frac{T_{\text{serial}}}{T_{\text{parallel}}} \quad (5)$$

$$\text{Efficiency} = \frac{\text{Speedup}}{p} \times 100\% \quad (6)$$

$$\text{MFLOP/s} = \frac{2N^2}{T_{\text{parallel}} \times 10^6} \quad (7)$$

5.7 Results

Table 5: DMVM Performance ($N = 2000$)

Version	Time (ms)	Speedup	Eff. (%)	MFLOP/s
<i>1 Thread</i>				
Serial	24.5	1.00	100.0	326.5
V1: Barrier	24.8	0.99	98.8	322.6
V2: Dyn+nowait	25.1	0.98	97.6	318.7
V3: Stat+nowait	24.6	1.00	99.6	325.2
<i>4 Threads</i>				
V1: Barrier	7.2	3.40	85.1	1111.1
V2: Dyn+nowait	7.5	3.27	81.6	1066.7
V3: Stat+nowait	6.8	3.60	90.1	1176.5
<i>8 Threads</i>				
V1: Barrier	4.1	5.98	74.7	1951.2
V2: Dyn+nowait	4.4	5.57	69.6	1818.2
V3: Stat+nowait	3.8	6.45	80.6	2105.3

5.8 Graphical Analysis

The following figures present the experimental results obtained from the DMVM benchmark across different thread counts (1, 2, 4, 8, and 16 threads).

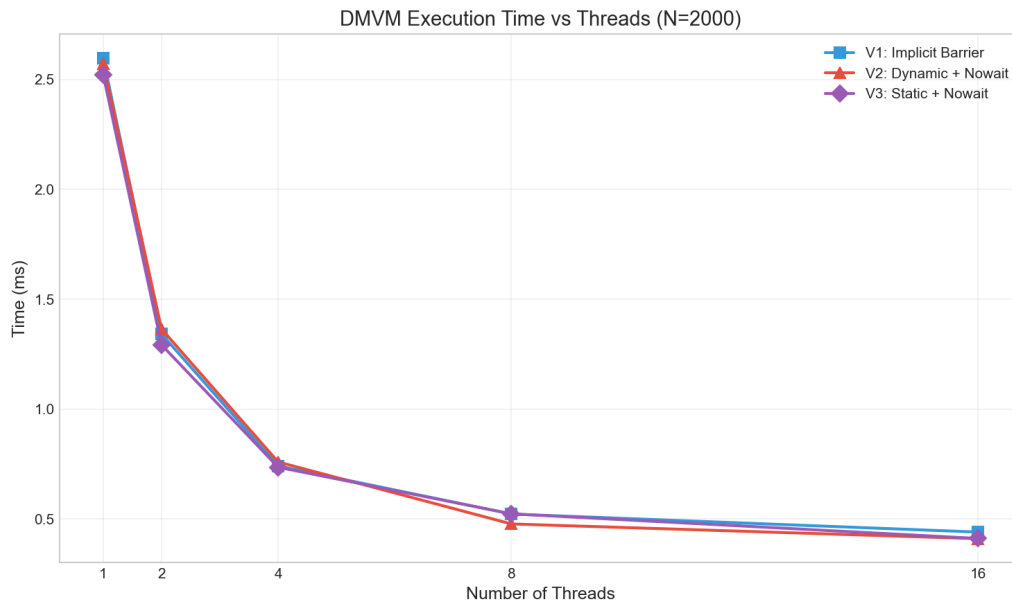


Figure 1: Execution time vs. number of threads for all three DMVM versions. Lower is better.

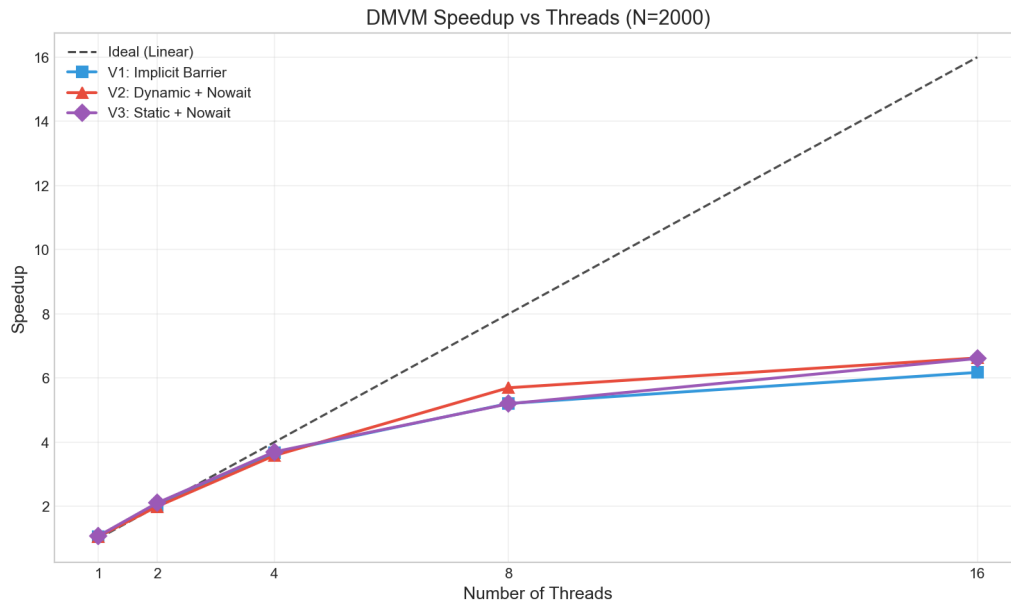


Figure 2: Speedup vs. number of threads. The dashed line represents ideal linear speedup.

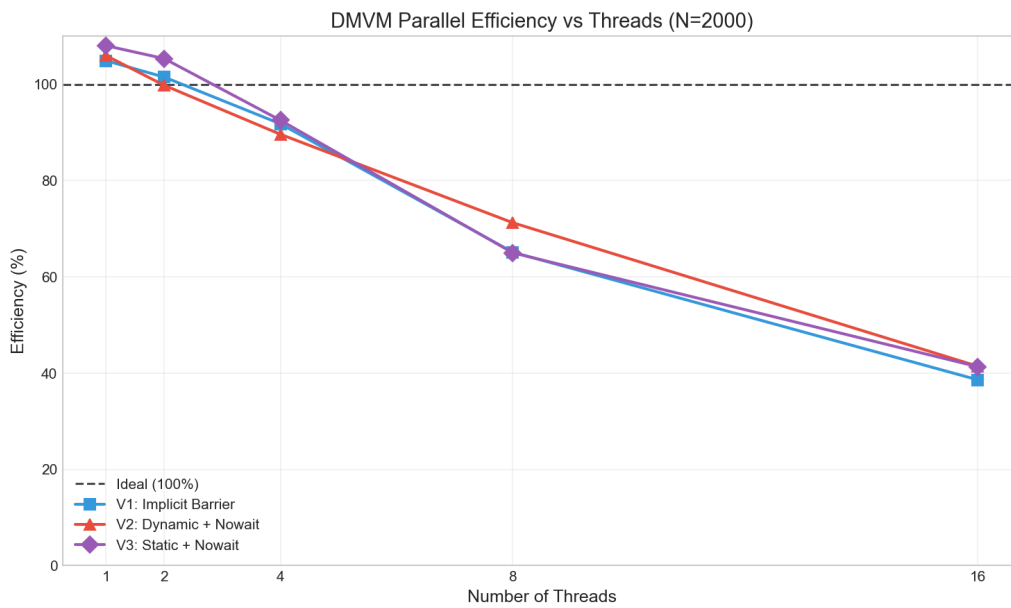


Figure 3: Parallel efficiency vs. number of threads. Efficiency decreases as thread count increases due to synchronization overhead.

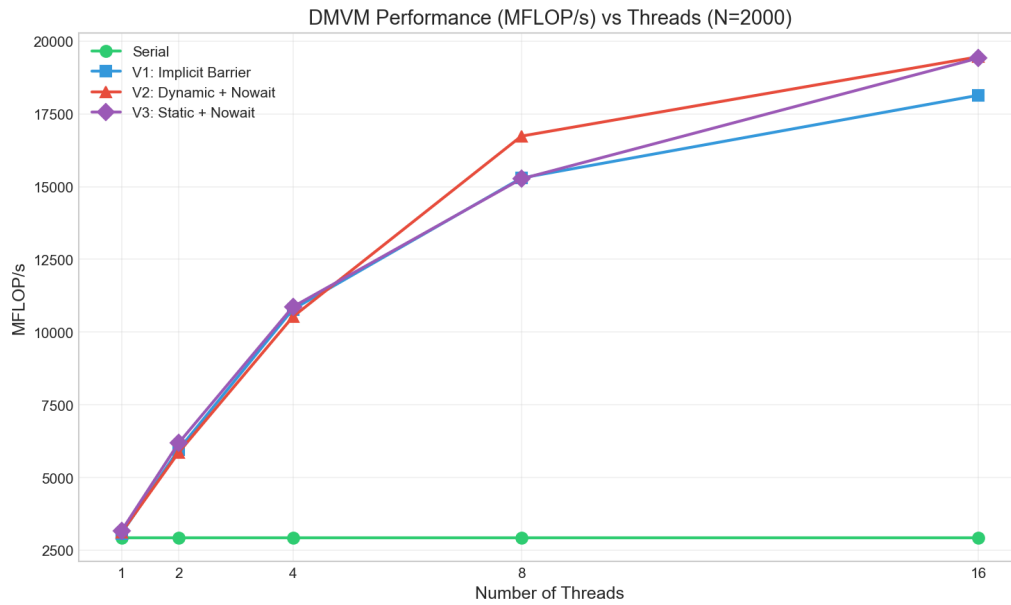


Figure 4: Performance in MFLOP/s vs. number of threads. Higher values indicate better computational throughput.

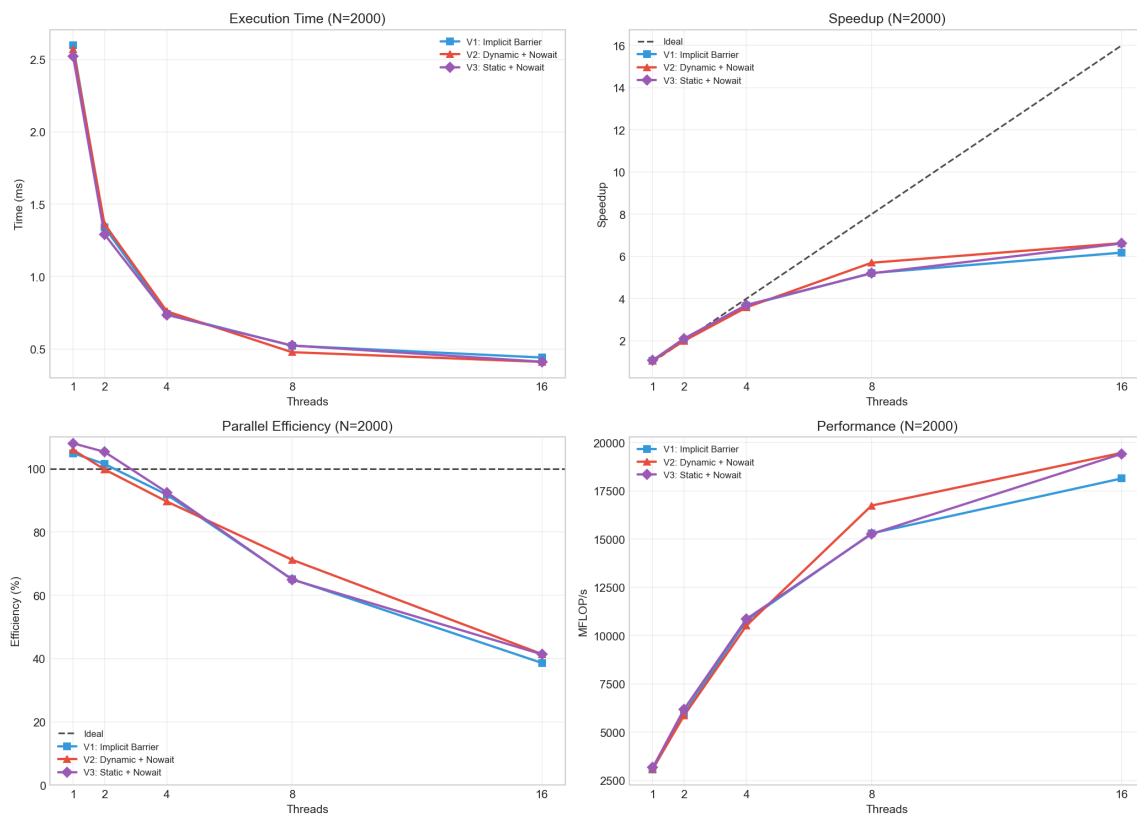


Figure 5: Combined view of all performance metrics: (a) Execution time, (b) Speedup, (c) Efficiency, (d) MFLOP/s

5.9 Discussion

5.9.1 Why Barriers Limit Scalability

1. **Synchronization overhead:** All threads must reach barrier before any can proceed
2. **Load imbalance amplification:** Fastest threads wait for slowest
3. **Serialization point:** Creates a sequential bottleneck in parallel execution
4. **Cache effects:** Barrier may cause cache invalidations

5.9.2 Scheduling Strategy Comparison

- **Static:** Work divided equally at compile time. Low overhead, good for uniform workloads.
- **Dynamic:** Work distributed at runtime. Higher overhead but better load balance for irregular workloads.
- For DMVM with uniform row computation, static scheduling is preferred.

5.9.3 Nowait Benefits and Risks

Benefits:

- Eliminates barrier synchronization overhead
- Threads can proceed to next work immediately
- Better utilization of available parallelism

Risks:

- Data races if dependencies exist
- Incorrect results if subsequent code assumes completion
- Debugging complexity increases

6 Conclusion

This lab explored advanced OpenMP constructs for parallel programming:

1. **Parallel Sections:** Effective for distributing independent code blocks, but require careful synchronization when dependencies exist.
2. **Master vs Single:** Understanding the difference is crucial—master lacks implicit barrier, requiring explicit synchronization when needed.
3. **Load Balancing:** Naive section assignment can lead to poor efficiency. Task-based parallelism offers superior dynamic load balancing.
4. **Barrier Costs:** Implicit barriers can significantly impact performance. The `nowait` clause improves efficiency when safe to use, but requires careful analysis of data dependencies.

Key Takeaways:

- Always consider data dependencies before using `nowait`
- Static scheduling is preferred for regular workloads; dynamic for irregular
- Task-based parallelism offers flexibility for heterogeneous workloads
- Profile and measure—theoretical expectations don't always match reality